



Extract API

Odoo provides a service to automate the processing of documents of type **invoices**, **bank statements**, **expenses** or **resumes**.

The service scans documents using an OCR engine and then uses AI-based algorithms to extract fields of interest such as the total, due date, or invoice lines for *invoices*, the initial and final balances, the date for *bank statements*, the total, date for *expenses*, or the name, email, phone number for *resumes*.

This service is a paid service. Each document processing will cost you one credit from your document digitization IAP account. More information about IAP accounts can be found [here](#).

You can either use this service directly in the Accounting, Expense, or Recruitment App or through the API. The Extract API, which is detailed in the next section, allows you to integrate our service directly into your own projects.

Overview

The extract API uses the [JSON-RPC2](#) protocol; its endpoint routes are located at `https://extract.api.odoo.com`.

Version

The version of the Extract API is specified in the route.

The latest versions are:

- invoices: 123
- bank statements: 100
- expenses: 132
- applicant: 102

Flow

The flow is the same for each document type.

1. Call [/parse](#) to submit your documents (one call for each document). On success, you receive a `document_token` in the response.

The HTTP POST method should be used for all of them. A python implementation of the full flow for invoices can be found [here](#) and a token for integration testing is provided in the [integration testing section](#).

Parse

Request the digitization of a document. The route will return a `document_token` that you can use to fetch the result of your request.

Routes

- `/api/extract/invoice/2/parse`
- `/api/extract/bank_statement/1/parse`
- `/api/extract/expense/2/parse`
- `/api/extract/applicant/2/parse`

Request

jsonrpc (required)

see [JSON-RPC2](#) 

method (required)

see [JSON-RPC2](#) 

id (required)

see [JSON-RPC2](#) 

params

account_token (required)

The token of the [IAP](#) account from which credits will be charged. Each successful call costs one credit.

version (required)

The version will determine the format of your requests and the format of the server response. You should use the [latest version available](#).

documents (required)

dbuuid (optional)

Unique identifier of the Odoo database.

webhook_url (optional)

A webhook URL can be provided. An empty POST request will be sent to `webhook_url/document_token` when the result is ready.

user_infos (optional)

Information concerning the person sending the document to the extract service. It can be the client or the supplier (depending on the `perspective`). This information is not required in order for the service to work but it greatly improves the quality of the result.

user_company_vat (optional)

VAT number of the user.

user_company_name (optional)

Name of the user's company.

user_company_country_code (optional)

Country code of the user. Format: [ISO3166 alpha-2](#).

user_lang (optional)

The user language. Format: *language_code* + `_` + *locale* (e.g. `fr_FR`, `en_US`).

user_email (optional)

The user email.

purchase_order_regex (optional)

Regex for purchase order identification. Will default to Odoo PO format if not provided.

perspective (optional)

Can be `client` or `supplier`. This field is useful for invoices only. `client` means that the user information provided are related to the client of the invoice. `supplier` means that it's related to the supplier. If not provided, `client` will be used.

```
{  
  "jsonrpc": "2.0",  
  "method": "call",  
  "params": {
```

```
        "webhook_url": string,
        "user_infos": {
            "user_company_vat": string,
            "user_company_name": string,
            "user_company_country_code": string,
            "user_lang": string,
            "user_email": string,
            "purchase_order_regex": string,
            "perspective": string,
        },
    },
    "id": string,
}
```

Note

The `user_infos` parameter is optional but it greatly improves the quality of the result, especially for invoices. The more information you can provide, the better.

Response

jsonrpc

see [JSON-RPC2](#)

id

see [JSON-RPC2](#)

result

status

The code indicating the status of the request. See the table below.

status_msg

A string giving verbose details about the request status.

document_token

Only present if the request is successful.

<code>error_unsupported_version</code>	Unsupported version
<code>error_internal</code>	An error occurred
<code>error_no_credit</code>	You don't have enough credit
<code>error_unsupported_format</code>	Unsupported file format
<code>error_maintenance</code>	Server is currently under maintenance, please try again later

```
{
  "jsonrpc": "2.0",
  "id": string,
  "result": {
    "status": string,
    "status_msg": string,
    "document_token": string,
  }
}
```

Note

The API does not actually use the JSON-RPC error scheme. Instead the API has its own error scheme bundled inside a successful JSON-RPC result.

Get results

Routes

- `/api/extract/invoice/2/get_result`
- `/api/extract/bank_statement/1/get_result`
- `/api/extract/expense/2/get_result`
- `/api/extract/applicant/2/get_result`

Request

jsonrpc (required)

see [JSON-RPC2](#) 

see [JSON-RPC2](#)

params

version (required)

The version should match the version passed to the [/parse](#) request.

document_token (required)

The `document_token` for which you want to get the current parsing status.

account_token (required)

The token of the `IAP` account that was used to submit the document.

```
{
  "jsonrpc": "2.0",
  "method": "call",
  "params": {
    "version": int,
    "document_token": int,
    "account_token": string,
  },
  "id": string,
}
```

Response

When getting the results from the parse, the detected field vary a lot depending on the type of document. Each response is a list of dictionaries, one for each document. The keys of the dictionary are the name of the field and the value is the value of the field.

jsonrpc

see [JSON-RPC2](#)

id

see [JSON-RPC2](#)

result

status

The code indicating the status of the request. See the table below.

results

Only present if the request is successful.

full_text_annotation

Contains the unprocessed full result from the OCR for the document.

status	status_msg
success	Success
error_unsupported_version	Unsupported version
error_internal	An error occurred
error_maintenance	Server is currently under maintenance, please try again later
error_document_not_found	The document could not be found
error_unsupported_size	The document has been rejected because it is too small
error_no_page_count	Unable to get page count of the PDF file
error_pdf_conversion_to_images	Couldn't convert the PDF to images
error_password_protected	The PDF file is protected by a password
error_too_many_pages	The document contains too many pages

```
{
  "jsonrpc": "2.0",
  "id": string,
  "result": {
    "status": string,
    "status_msg": string,
    "results": [
      {
        "full_text_annotation": string,
        "feature_1_name": feature_1_result,
        "feature_2_name": feature_2_result,
        ...
      }
    ]
  }
}
```

```
}

```

Common fields

feature_result

Each field of interest we want to extract from the document such as the total or the due date are also called **features**. An exhaustive list of all the extracted features associated to a type of document can be found in the sections below.

For each feature, we return a list of candidates and we spotlight the candidate our model predicts to be the best fit for the feature.

selected_value (optional)

The best candidate for this feature.

selected_values (optional)

The best candidates for this feature.

candidates (optional)

List of all the candidates for this feature ordered by decreasing confidence score.

```
"feature_name": {
    "selected_value": candidate_12,
    "candidates": [candidate_12, candidate_3, candidate_4, ...]
}
```

candidate

For each candidate we give its representation and position in the document. Candidates are sorted by decreasing order of suitability.

content

Representation of the candidate.

coords

[center_x, center_y, width, height, rotation_angle] . The position and dimensions are relative to the size of the page and are therefore between 0 and 1. The angle is a clockwise rotation measured in degrees.

```
"candidate": [
    {
        "content": string|float,
        "coords": [float, float, float, float, float],
        "page": int
    },
    ...
]
```

Invoices

Invoices are complex and can have a lot of different fields. The following table gives an exhaustive list of all the fields we can extract from an invoice.

Feature name	Specificities
SWIFT_code	<code>content</code> is a dictionary encoded as a string. It contains information about the detected SWIFT code (or BIC . Keys: bic detected BIC (string). name (optional) bank name (string). country_code ISO3166 alpha-2 country code of the bank (string). city (optional) city of the bank (string). verified_bic True if the BIC has been found in our DB (bool). Name and city are present only if <code>verified_bic</code> is true.
iban	<code>content</code> is a string

VAT_Number	content is a string
	Depending on the value of perspective in the user_infos, this will be the VAT number of the supplier or the client. If perspective is client, it'll be the supplier's VAT number. If it's supplier, it's the client's VAT number.
qr-bill	content is a string
payment_ref	content is a string
purchase_order	content is a string
	Uses <code>selected_values</code> instead of <code>selected_value</code>
country	content is a string
currency	content is a string
date	content is a string
	Format : <i>YYYY-MM-DD</i>
due_date	Same as for <code>date</code>
total_tax_amount	content is a float
invoice_id	content is a string
subtotal	content is a float
total	content is a float
supplier	content is a string
client	content is a string
email	content is a string
website	content is a string

invoice_lines feature

```
{
    "description": string,
    "quantity": float,
    "subtotal": float,
    "total": float,
    "taxes": list[float],
    "total": float,
    "unit_price": float
},
...
]
```

Bank statements

The following table gives a list of all the fields that are extracted from bank statements.

Feature name	Specificities
balance_start	content is a float
balance_end	content is a float
date	content is a string

bank_statement_lines feature

It is returned as a list of dictionaries where each dictionary represents a bank statement line.

```
"bank_statement_lines": [
    {
        "amount": float,
        "description": string,
        "date": string,
    },
    ...
]
```

Expense

Feature name	Specificities
--------------	---------------

description	content is a string
-------------	---------------------

country	content is a string
---------	---------------------

date	content is a string
------	---------------------

total	content is a float
-------	--------------------

currency	content is a string
----------	---------------------

Applicant

This third type of document is meant for processing resumes. The following table gives an exhaustive list of all the fields we can extract from a resume.

Feature name	Specificities
--------------	---------------

name	content is a string
------	---------------------

email	content is a string
-------	---------------------

phone	content is a string
-------	---------------------

mobile	content is a string
--------	---------------------

Integration Testing

You can test your integration by using *integration_token* as `account_token` in the [/parse](#) request.

Using this token put you in test mode and allows you to simulate the entire flow without really parsing a document and without being billed one credit for each successful **document** parsing.

The only technical differences in test mode is that the document you send is not parsed by the system and that the response you get from [/get_result](#) is a hard-coded one.

A python implementation of the full flow for invoices can be found [here](#) .



Get Help

Contact Support

Ask the Odoo Community

